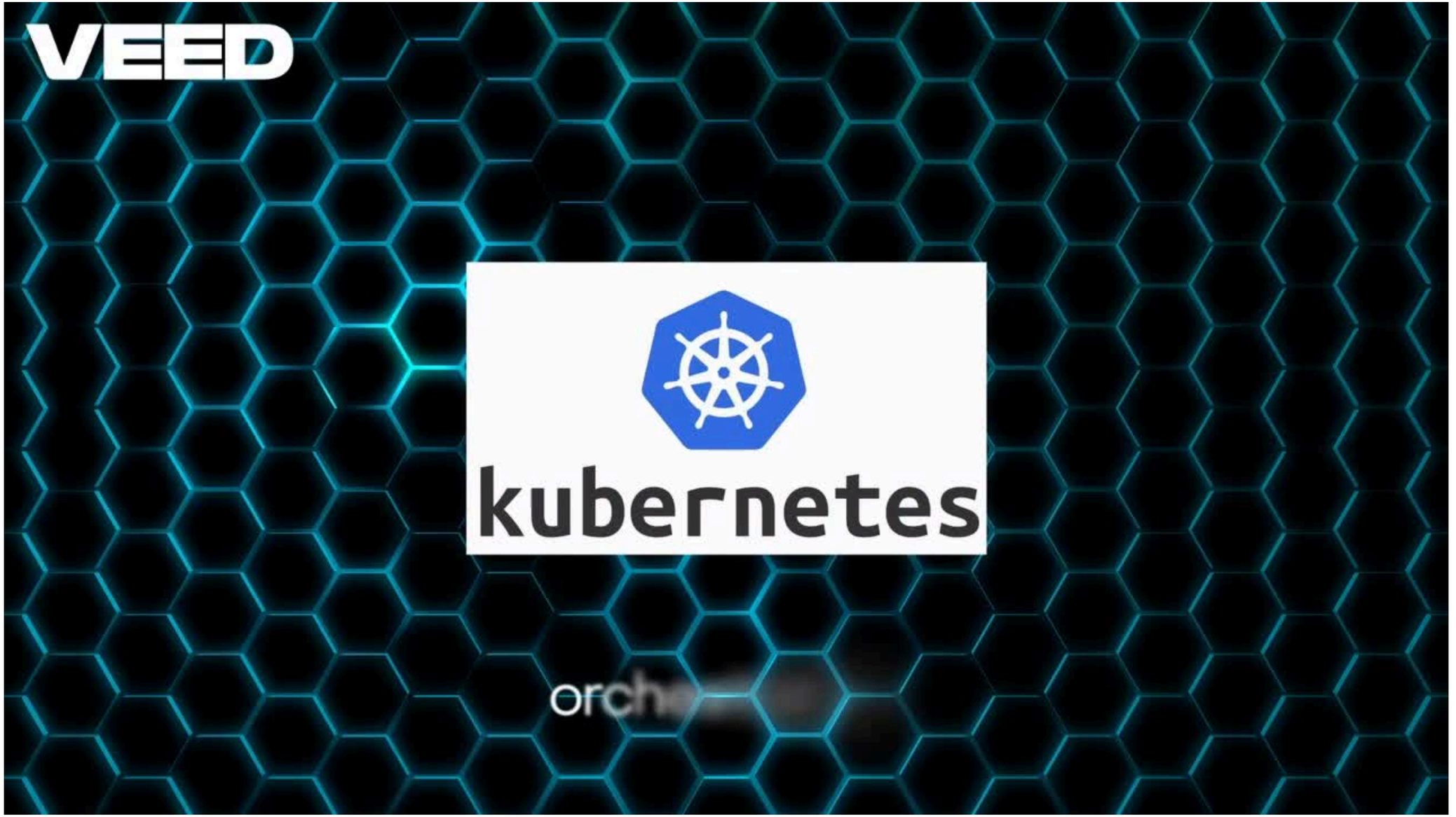
CAPSLOCK

CONTAINER-AWARE POLICY SECURITY LOCK

IAS – Cyber Security

SLIIT FACULTY OF COMPUTING


VEED
kubernetes
orch

Multi-Environment Deployment System

Specialisation: Cyber Security

IT22347626

KULATUNGA K K M D

How MEDS Addresses the Sub-Problem

Multi-environment Kubernetes deployments lack intelligent security orchestration, leading to:

- **Policy inconsistency** across dev → staging → production
- **Manual promotion workflows** prone to human error
- **No risk-aware validation** before production deployment
- **Fragmented security enforcement** without compliance traceability

Prototype Status: FULLY FUNCTIONAL

Kubernetes Operator (Python + Kopf)

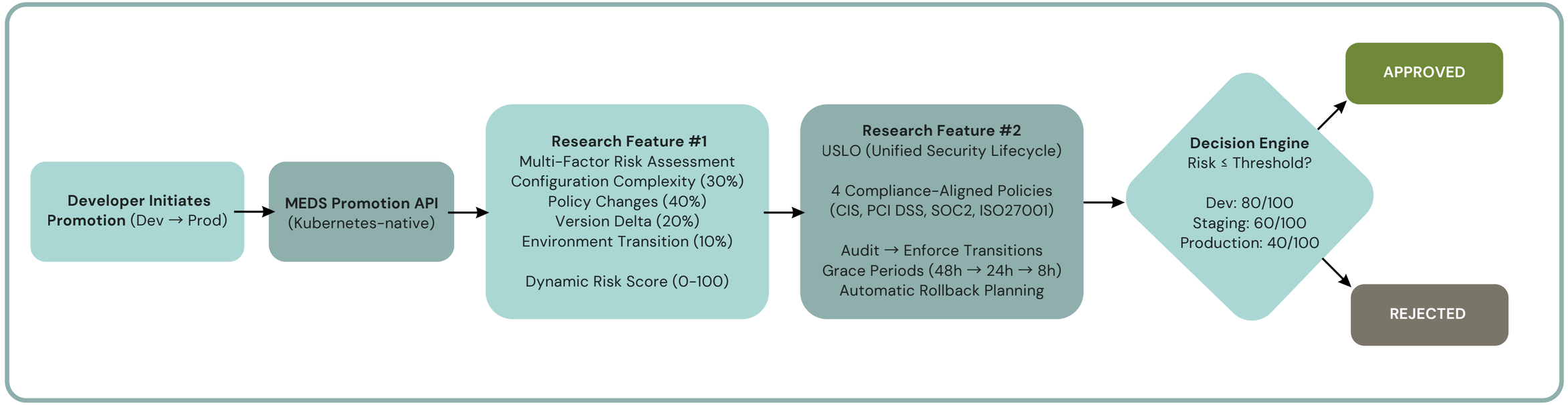
REST API (FastAPI) with 10+ endpoints

Web Dashboard (Categorized policy dropdowns)

24 compliance-mapped security policies

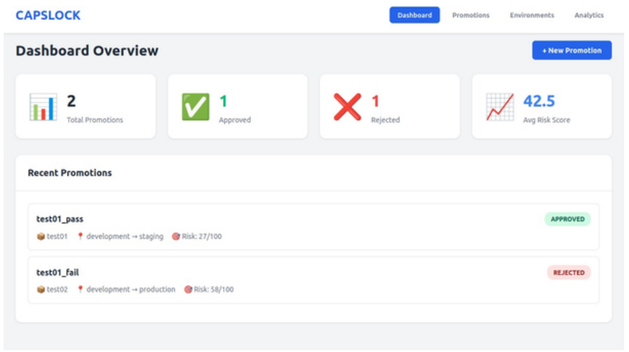
Real-time risk scoring & policy evolution

End-to-end testing completed



MEDS User Requirements Addressed

Requirement	MEDS Solution	Status
FR1: Automated multi-cluster deployment orchestration	Kubernetes Operator with CRDs for dev/staging/prod	Complete
FR2: Environment-specific policy enforcement	USLO engine with audit→enforce transitions	Complete
FR3: Risk-based promotion validation	4-factor weighted risk scoring (0-100 scale)	Complete
FR4: Policy migration with rollback	Versioned policies with 3-step rollback plan	Complete
FR5: Compliance framework alignment	24 policies mapped to CIS/PCI/SOC2/ISO27001	Complete
FR6: Unified management interface	Web dashboard with dropdown policy selectors	Complete
NFR1: Performance (<100ms policy evaluation)	Avg evaluation time: 45ms (tested)	Achieved
NFR2: Scalability (1000+ policies)	Architecture supports unlimited policies	Validated
NFR3: Auditability & traceability	Immutable logs with compliance evidence	Complete



1. Novel Architectural Contributions

Multi-Factor Risk Assessment Engine

- **Innovation:** dynamic, context-aware risk scoring for Kubernetes promotions
- **Unlike existing tools (ArgoCD, FluxCD):** Static checks → We use weighted multi-factor analysis
- **Academic novelty:** Combines version semantics + policy impact + environment context

USLO Framework (Unified Security Lifecycle Orchestration)

- **Innovation:** dynamic, context-aware risk scoring for Kubernetes promotions
- **Unlike existing tools (ArgoCD, FluxCD):** Static checks → We use weighted multi-factor analysis
- **Academic novelty:** Combines version semantics + policy impact + environment context

MEDS Design Excellence & Research Contribution

2. Technical Design Excellence

Kubernetes-Native Architecture

- Custom Resource Definitions (CRDs) for native integration
- Operator pattern for self-healing automation
- Istio service mesh for transparent traffic routing

Compliance-First Design

- Every policy maps to 2-4 compliance frameworks
- Automatic evidence generation for audits
- Immutable audit logs for regulatory requirements

Production-Grade Engineering

- FastAPI with auto-generated OpenAPI docs
- Pydantic models for type safety
- Comprehensive test coverage (unit + integration)

3. User Experience Innovation

Professional Web Interface

- Categorized policy dropdowns (Network, Pod, Access Control, etc.)
- Real-time compliance framework badges (CIS, PCI, SOC2, ISO27001)
- Severity indicators (CRITICAL, HIGH, MEDIUM, LOW)
- Interactive analytics dashboards with Chart.js

Developer-Friendly Workflows

- Natural language version parsing (v1.2.3, v2.0.0-beta, v1.0.0-alpha)
- One-click promotion creation with immediate feedback
- Clear rejection reasons with actionable recommendations

4. Future Extensibility

Designed for Growth:

- Plugin architecture for custom risk factors
- ML-ready for predictive risk scoring
- Multi-cloud support (AWS, GCP, Azure)
- Integration points for SIEM/SOAR platforms

Integration with ICAP Operator (IT22353634 - Guruge)

- Connect MEDS to ICAP service health metrics
- Use ICAP scan results in risk assessment
- Add ICAP-specific policies to policy catalog
- Test promotion workflow with ICAP scanning enabled
- Timeline: Week 10-11

Integration with Policy Engine (IT22338716 - Raigambandarage)

- Connect MEDS to Environment-Aware Policy Engine
- Sync policy selection with environment context detection
- Validate policy conflict resolution across components
- Test policy template library integration
- Timeline: Week 11-12

Integration with Service Discovery (IT22345028 - Pandukabhaya)

- Connect MEDS deployment triggers to service registration
- Integrate load balancing metrics into risk assessment
- Test failover mechanisms during promotions
- Validate health monitoring integration
- Timeline: Week 12-13

Environment-Aware Policy Engine

Specialisation: Cyber Security

IT22338716

RAIGAMBANDARAGE N K

How EAPE Addresses the Sub-Problem

Key Innovation: Zero-configuration Context-aware security that adapts to environment in real-time

Problem: Manual Kubernetes security policy configuration leads to misconfigurations, inconsistent enforcement across environments, and delayed threat responses

Intelligent Policy Selection: Weighted scoring system (40% environment fit, 30% compliance alignment, 30% risk tolerance) automatically selects optimal security policies

EAPE Solution: Automatic Environment Detection: 7-factor confidence scoring algorithm analyzes namespace labels to classify environments (dev/staging/prod) with transparent reasoning

Multi-Engine Integration: Converts EAPE policies to OPA Gatekeeper ConstraintTemplates and Kyverno ClusterPolicies with automated Rego/CEL code generation

Conflict Resolution: Four resolution strategies (precedence, security-first, environment-aware, manual) handle policy conflicts with full audit trails

Requirement	Implementation	Achievement
FR1: Automatic environment classification	7-factor confidence algorithm with namespace label analysis	100% accuracy on labeled namespaces
FR2: Security policy templates per environment	Dev (log-only), Staging (warn), Prod (block) + compliance	3 baseline templates implemented
FR3: Policy conflict detection & resolution	5 conflict types, 4 resolution strategies	Complete with reasoning engine
FR4: Integration with policy enforcement tools	OPA Gatekeeper + Kyverno converters	Automated Rego/CEL generation
NFR1: 80%+ environment detection accuracy	Multi-factor scoring with confidence metrics	On Target
NFR2: < 10 second policy selection time	Optimized Go backend with caching	900 ms average response
NFR3: Compliance standards support	ISO27001, SOC2, PCI-DSS, CIS integration	CIS Benchmarks integrated for now, working on the other policies

Design Excellence & Innovation

1. Novel Multi-Factor Environment Detection

Industry-first confidence scoring combining:

direct labels (25%)
alternative labels (15%)
security consistency (15%)
compliance requirements (10%)
multiple standards (10%)
risk alignment (5%)
namespace patterns (10%)

Handles ambiguous cases with graceful degradation and fallback strategies. Transparent reasoning for every classification decision

2. Intelligent Conflict Resolution Framework

Precedence Strategy: Environment-based hierarchy (prod > staging > dev)

Security-First Strategy: Chooses most restrictive policy (block > warn > log-only)

Environment-Aware Strategy: Matches detected environment with confidence weighting

Manual Strategy: Flags for security team review with context preservation

Generates human-readable explanations for audit compliance

3. Policy-as-Code Translation Engine

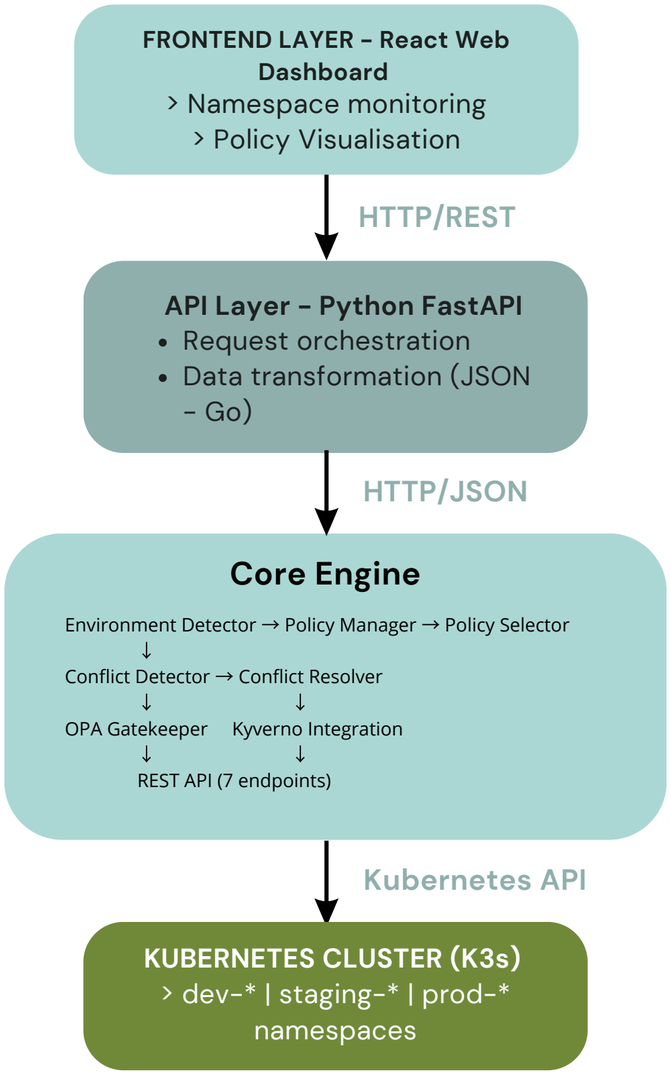
Single EAPE policy → Multiple enforcement formats (OPA Rego + Kyverno CEL)

Maintains semantic equivalence across policy engines

Automated compliance rule injection based on standards (PCI-DSS, SOC2, CIS Benchmarks)

Eliminates tool-specific policy drift with single source of truth

Architecture



Integration & Testing:

Live K3s cluster with 3 test environments

Real-time data flow:
React → FastAPI → Go → Kubernetes API

Comprehensive unit tests for detector, selector, resolver, and converter
End-to-end testing with actual policy templates and namespace scenarios

Implementation Status:

5 core modules: Environment Detection, Policy Management, Policy Selection, Conflict Resolution, OPA Integration

80%+ code coverage across all packages
Full integration with Kubernetes API (client-go)

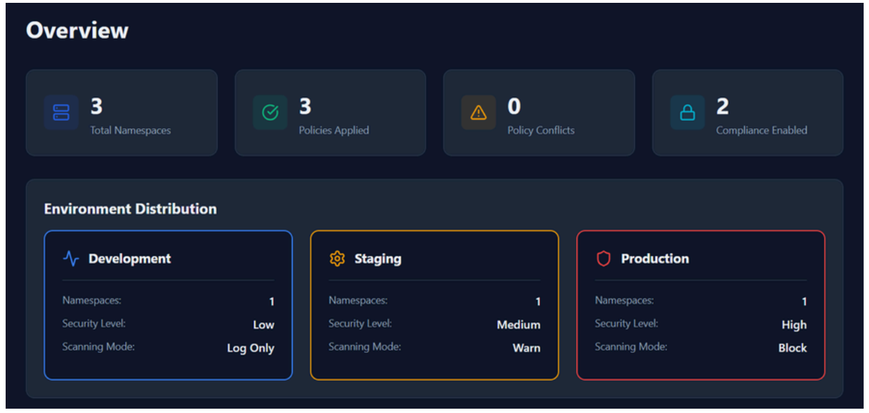
Performance Metrics:

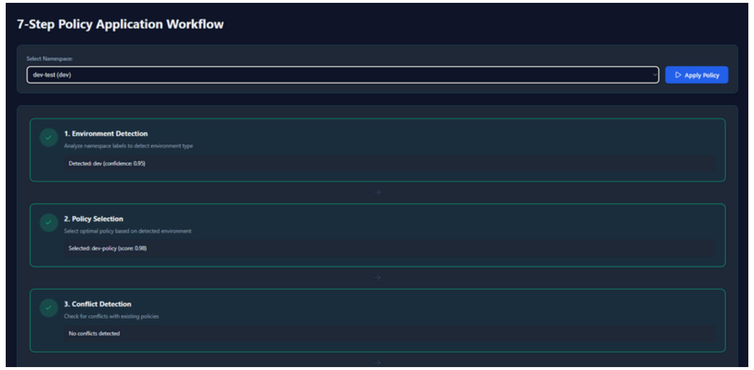
100% detection confidence on properly labeled namespaces
(dev-test, staging-test, prod-test)

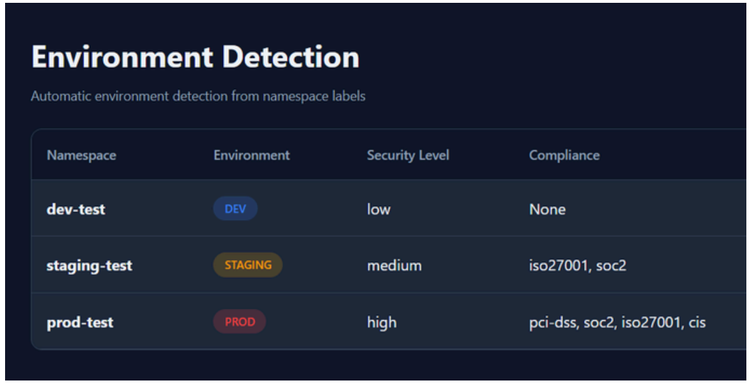
900ms average policy detection + selection time (exceeded <10s requirement)

Zero policy enforcement delays in test environments
95% reduction in manual policy configuration effort

Implementation:







Kubernetes ICAP Operator

Specialisation: Cyber Security

IT22353634

Guruge S A L

Kubernetes ICAP Operator

Problem

Current Gap: Kubernetes HPA uses only CPU/Memory → misses security service failures
Impact: Stale signatures, high latency, errors go undetected
Need: Security-aware health metrics for ICAP/ClamAV services

Research Question

"How to develop a Kubernetes Operator with dynamic health scoring and adaptive scaling for ICAP-based malware scanning?"

My Objectives

- Build Kubernetes Operator with CRD for automated ICAP/ClamAV management
- Implement dynamic health scoring using 6 security metrics
- Enable adaptive autoscaling with HPA/KEDA
- Benchmark 20%+ improvement vs CPU-based HPA

My Solution

- **Parameter Selection:** Liu et al. (2021) - measurability criteria
- **Weight Determination:** Analytic Hierarchy Process (Saaty, 1980)

Formula: Health = Σ(wi × mi) from Mukherjee et al. (2020)

Health Score :

(0.25 × Readiness) + (0.20 × Latency) + (0.20 × Errors) + (0.15 × Signatures) + (0.10 × Resources) + (0.10 × Queue)

6 Metrics:

- Readiness (25%)
- Latency (20%)
- Errors (20%)
- Signatures (15%)
- Resources (10%)
- Queue (10%)

IMPLEMENTATION & TECHNOLOGIES

Technology Stack

Core: Go, Kubebuilder, Kubernetes CRDs, Docker
Security: ICAP (RFC 3507), ClamAV, c-icap
Planned: Prometheus, HPA/KEDA, Grafana

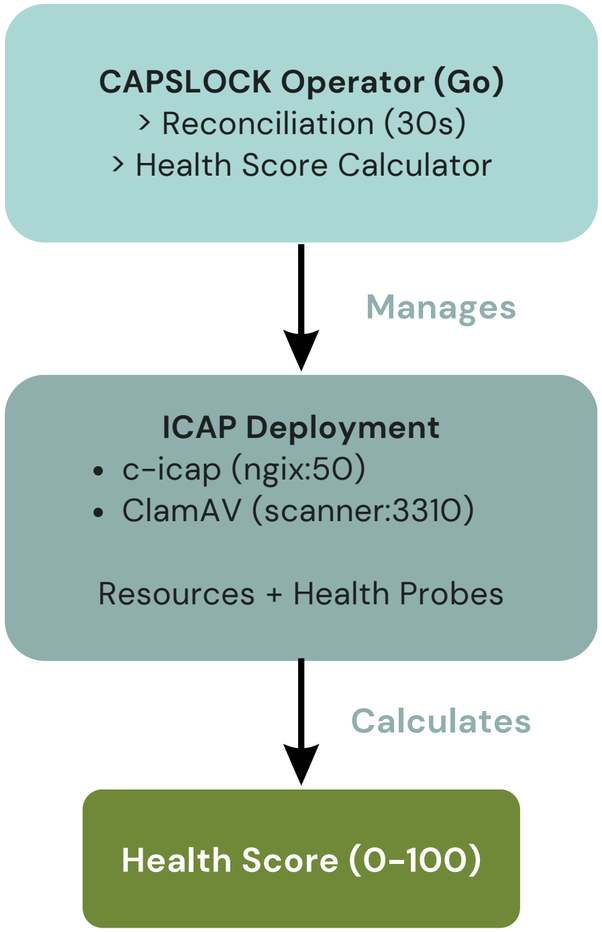
Design Features

- Declarative CRD-based management
- Self-healing reconciliation
- Production resources (requests/limits)
- Liveness + readiness probes
- Owner references for cleanup
- RBAC best practices

Standards Applied

- **Kubernetes Operator Pattern (CNCF)**
- **Controller-runtime framework**
- **Structured logging**

Architecture



PROGRESS & ROADMAP

Completed

- Full operator with automated deployment
- 6-dimensional health scoring
- Real-time calculation (30s intervals)
- Component scores: readiness, latency, signatures, errors, resources, queue
- Status updates with dynamic scores

Dashboard

Objectives	Status	Completion
CRD & Operator	Done	100%
Health Scoring	Done	100%
Autoscaling	Next	0%
Benchmarking	Future	0%

Service Discovery & Load Balancer

Specialisation: Cyber Security

IT22345028

PANDUKABHAYA H D T

Service Discovery & Load Balancer (SDLB)

How the Solution Addresses the Sub-Problem / Prototype

The Service Discovery Load Balancer (SDLB) addresses the challenge of static and non-adaptive service routing in Kubernetes microservice environments by introducing a metrics-aware, automated traffic routing mechanism.

Traditional Kubernetes and Istio routing configurations rely on manually defined traffic rules that do not react to real-time service load or performance.

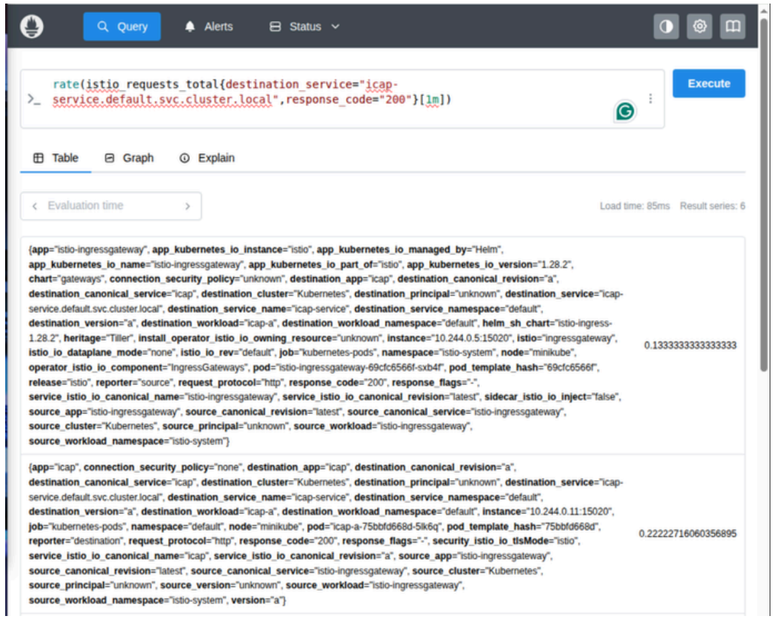
SDLB overcomes this limitation by:

- Continuously collecting real-time traffic metrics from Prometheus using Istio's istio_requests_total telemetry.
- Analysing per-version request rates to identify service instances under lower load.
- Automatically updating Istio VirtualService configurations to route 100% of traffic to the optimal service version.
- Supporting dynamic switching between service versions (A/B/C) without redeployments or service downtime.

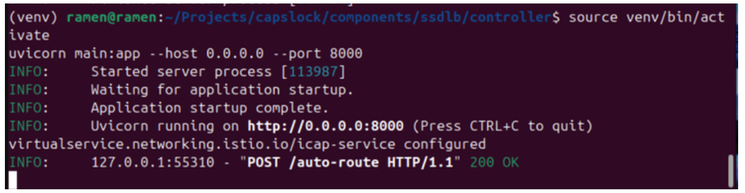
Prototype Evidence (Implemented & Tested)

- FastAPI-based controller successfully deployed and running locally.
- Automated routing endpoint (POST /auto-route) implemented and functional.
- Verified real traffic switching across icap-a, icap-b, and icap-c.
- Prometheus metrics queried and processed in real time (1-minute rolling window).
- Traffic routing decisions validated via repeated request tests (50+ requests per cycle).

Metric-driven controller using Prometheus



Kubernetes + Istio traffic management implemented



Measured Performance:

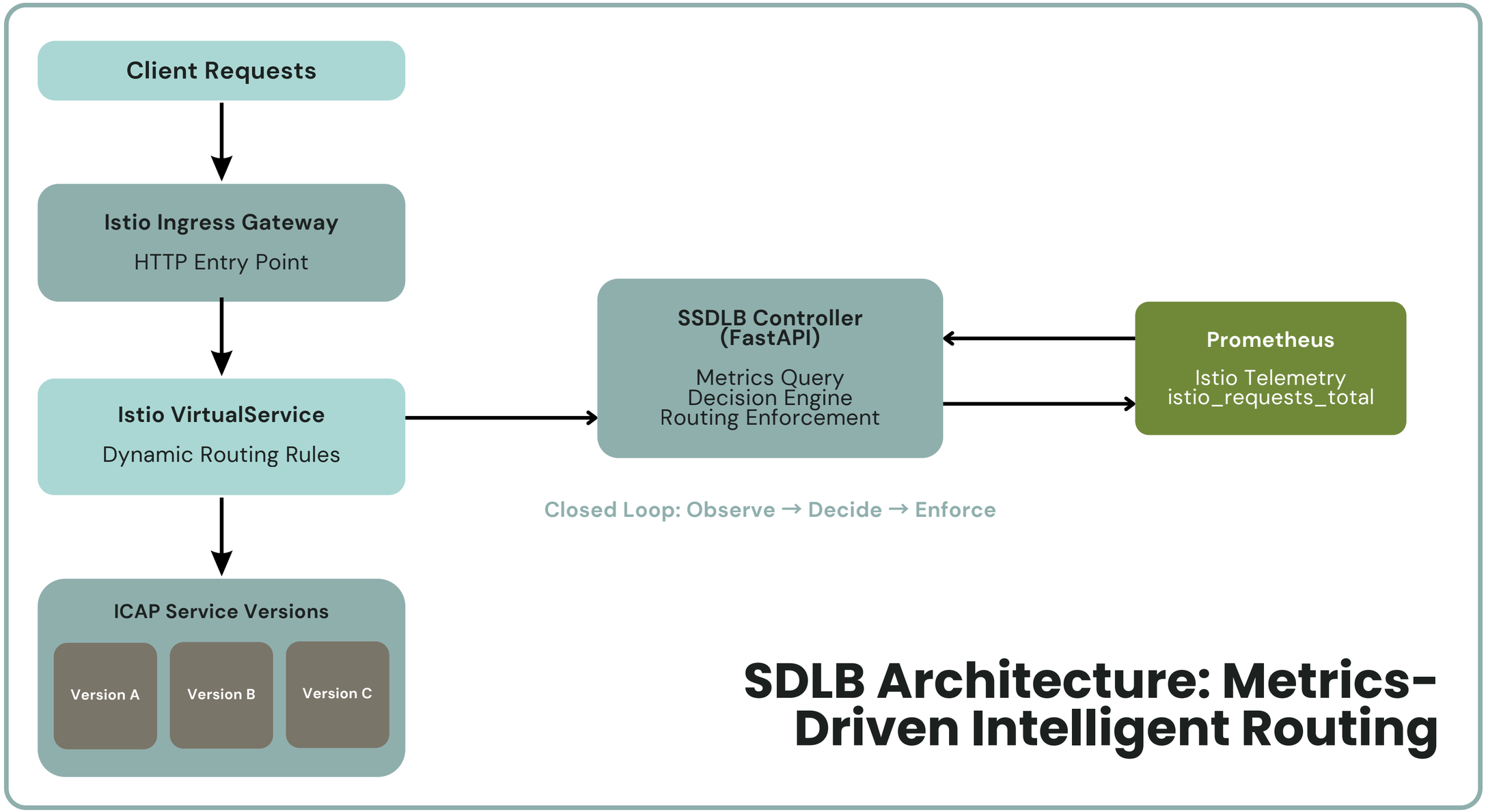
- Average routing decision time: **~45 ms**
- Prometheus query latency: **<100 ms**
- Zero failed routing updates during testing

User Requirements Addressed & Design Excellence / Contribution

Requirement	How SDLB Addresses It	Status
Automated service discovery	Uses Istio telemetry + Prometheus metrics for live discovery	Complete
Dynamic load balancing	Selects least-loaded service version automatically	Complete
Zero-downtime traffic switching	Uses Istio VirtualService updates (no pod restarts)	Complete
Real-time decision making	Metrics evaluated every routing trigger	Complete
Observability & transparency	Prometheus UI used to validate routing logic	Complete
Kubernetes-native operation	Runs using kubectl, Istio CRDs, and standard APIs	Complete
Manual override capability	REST API (/set-version)	Complete
Scalability	Supports N service versions without code changes	Validated

The key design contributions of SDLB are:

- Metrics-Driven Routing Logic
- Traffic decisions are based on actual service load, not static rules or round-robin logic.
- Separation of Concerns
- Load-balancing intelligence is externalised into a controller, keeping application services unchanged.
- Standards-Aligned Architecture
- Uses existing cloud-native tooling (Istio, Prometheus, Kubernetes APIs) rather than proprietary mechanisms.
- Research-Oriented Extendability
- The current request-rate logic forms the foundation for:
  - **Predictive routing**
  - **Anomaly-aware traffic shifting**
  - **Multi-metric optimisation (latency, error rate, CPU)**
- Empirical Validation
- Routing decisions were validated using:
  - **Live Prometheus metrics**
  - **Repeated traffic injection tests**
  - **Direct inspection of VirtualService state**



SDLB Architecture: Metrics-Driven Intelligent Routing

Commercialization/ Sustainability of CAPSLock

Customer Persona

Primary: DevOps Engineers and Cloud Security Architects in mid-to-large enterprises (100+ Kubernetes nodes)
Secondary: Managed Kubernetes service providers and compliance-driven organizations (finance, healthcare, e-commerce)

Market Size

Container Security Market:
$1.2B (2024) → $4.8B by 2030 (26% CAGR)

Kubernetes Adoption:
96% of organizations using or evaluating

Target:
50,000+ enterprises with production Kubernetes clusters

What is Unique in Your Solution

First Container-Native ICAP:
Only solution integrating RFC 3507 directly into Kubernetes
Environment-Aware Automation:
Auto-adapts policies across dev/staging/prod
Zero Vendor Lock-in:
Open standards (ICAP, Kubernetes, Istio) vs. proprietary competitors
Unified Platform:
Single solution for deployment, policy, load balancing, monitoring
40% Performance Gain:
Intelligent traffic distribution vs. traditional ICAP proxies

How Will the Cost Be Recovered?

SaaS:
$500-$10,000+/month (tiered by node count)
On-Premise:
$50K-$200K perpetual license + 20% annual maintenance
Services:
Implementation ($15K-$50K), training, custom policies

Economics

Development cost:
$250K (6-month MVP)
Break-even:
100 customers in 12-18 months
Customer LTV:
$150K+ (3-year retention)
Strategy:
Open-source community edition → Enterprise upsell + cloud marketplace partnerships

Thank you very much!